Sometimes these class-level methods return collections of objects:

```
Order.where(name: 'dave').each do |order|
  puts order.amount
end
```

Finally, the objects corresponding to individual rows in a table have methods that operate on that row. Probably the most widely used is `save`, the operation that saves the row to the database:

```
Order.where(name: 'dave').each do |order|
  order.pay_type = "Purchase order"
  order.save
end
```

So an ORM layer maps tables to classes, rows to objects, and columns to attributes of those objects. Class methods are used to perform table-level operations, and instance methods perform operations on the individual rows.

In a typical ORM library, you supply configuration data to specify the mappings between entities in the database and entities in the program. Programmers using these ORM tools often find themselves creating and maintaining a boatload of XML configuration files.

Active Record

Active Record is the ORM layer supplied with Rails. It closely follows the standard ORM model: tables map to classes, rows to objects, and columns to object attributes. It differs from most other ORM libraries in the way it's configured. By relying on convention and starting with sensible defaults, Active Record minimizes the amount of configuration that developers perform.

To show this, here's a program that uses Active Record to wrap our `orders` table:

```
require 'active_record'

class Order < ApplicationRecord
end

order = Order.find(1)
```